Deploy Manufacturing App to Kubernetes Report

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Om Tiwari	EN22CS301669
02	Paridhi Shirwalkar	EN22CS301684
03	Nitesh Chourasiya	EN22CS301660
04	Mradul Jain	EN22CS301616

Group Name: 01D6

Project Number: DO-12

Industry Mentor Name: Vaibhav

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-2026

Table of Contents

1. **Problem Statement & Objectives**
   - 1.1 Problem Statement
   - 1.2 Project Objectives
   - 1.3 Scope of the Project
2. **Proposed Solution**
   - 2.1 Key Features
   - 2.2 Overall Architecture / Workflow
   - 2.3 Tools & Technologies Used
3. **Results & Output**
   - 3.1 Screenshots / Outputs
   - 3.2 Reports / Dashboards
   - 3.3 Key Outcomes
4. **Conclusion**
5. **Future Scope & Enhancements**
6. **References**
7. **GitHub**

1. Problem Statement & Objectives

1.1 Problem Statement

Modern software applications require reliable, scalable, and automated deployment infrastructure to ensure continuous availability and seamless updates. The Multi-Agent Manufacturing System, built with Python, Streamlit, and CrewAI, needed a production-grade deployment pipeline that could containerize the application, manage it on a cloud Kubernetes cluster, and deliver new versions without any service interruption.

1.2 Project Objectives

- Containerize the Multi-Agent Manufacturing System using Docker with a clean Linux-compatible build strategy
- Deploy the application on AWS EKS Kubernetes cluster in the Mumbai region
- Implement zero-downtime Rolling Updates ensuring the app remains available during every deployment
- Build a fully automated GitHub Actions CI/CD pipeline that deploys on every git push with no manual steps
- Manage all credentials securely using GitHub Secrets with no sensitive data stored in code or images

1.3 Scope of the Project

This project covers the complete DevOps infrastructure for deploying the Multi-Agent Manufacturing System. It includes Docker containerization, Kubernetes deployment and service configuration, AWS EKS cluster setup, CI/CD pipeline implementation using GitHub Actions, AWS Load Balancer configuration for public access, and security through runtime secret management. The project does not cover the internal AI logic of the CrewAI agents or the business logic of the manufacturing domain those are addressed in the Multi-agent-system repository.

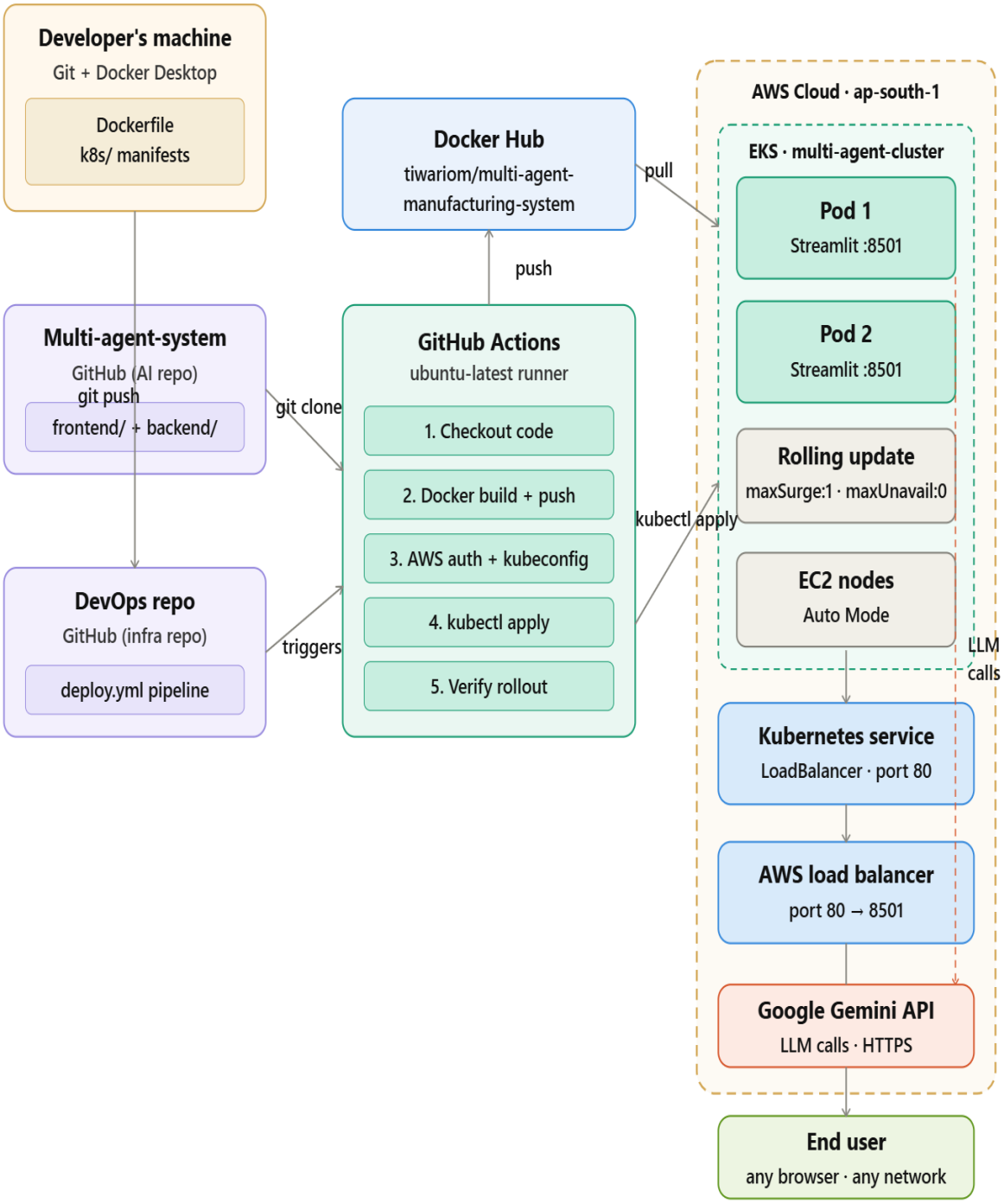
2.Proposed Solution

2.1 Key Features

- Automated CI/CD pipeline triggered on every git push to main branch
- Zero-downtime Rolling Updates using Kubernetes deployment strategy
- Docker containerization with GitHub-based build no local builds needed
- Two-repository architecture DevOps repo contains zero application code
- Deployed on AWS EKS with auto-provisioned nodes and public Load Balancer

- Health-check-driven deployments using Streamlit's built-in /_stcore/health endpoint
- All credentials managed via GitHub Secrets nothing stored in code or images

2.2 Overall Architecture / Workflow

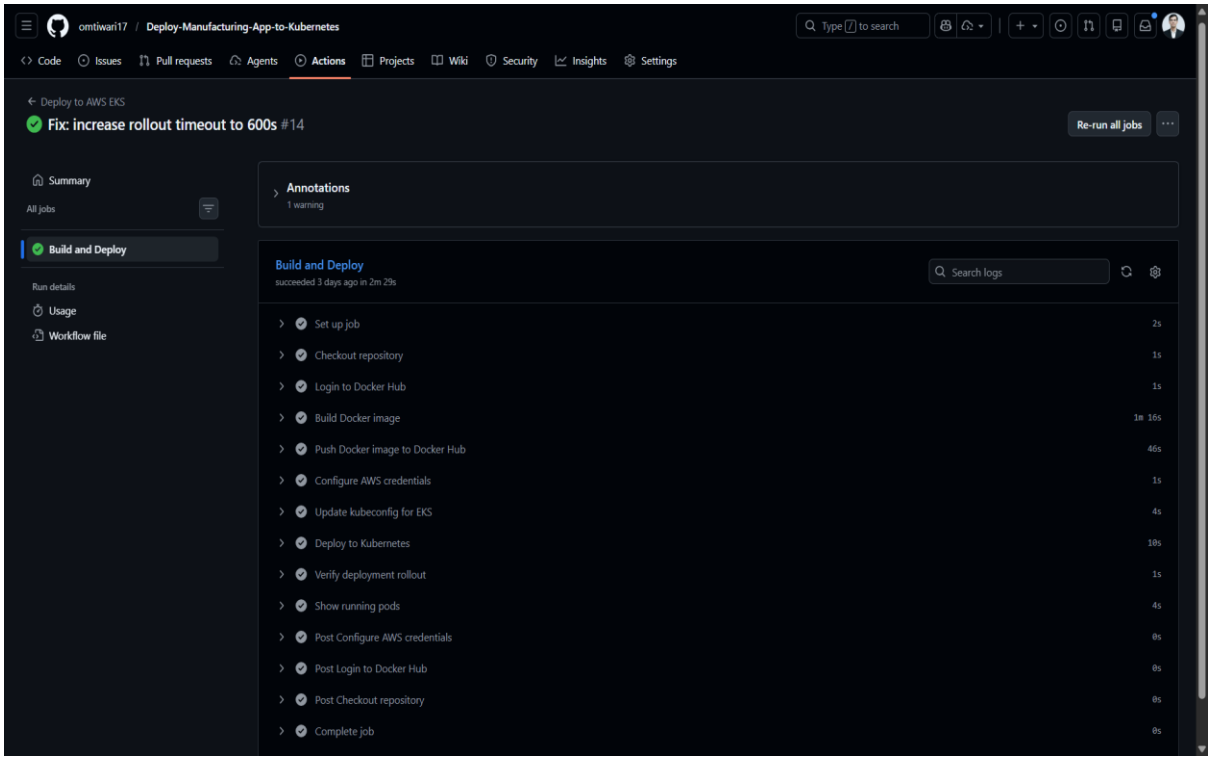


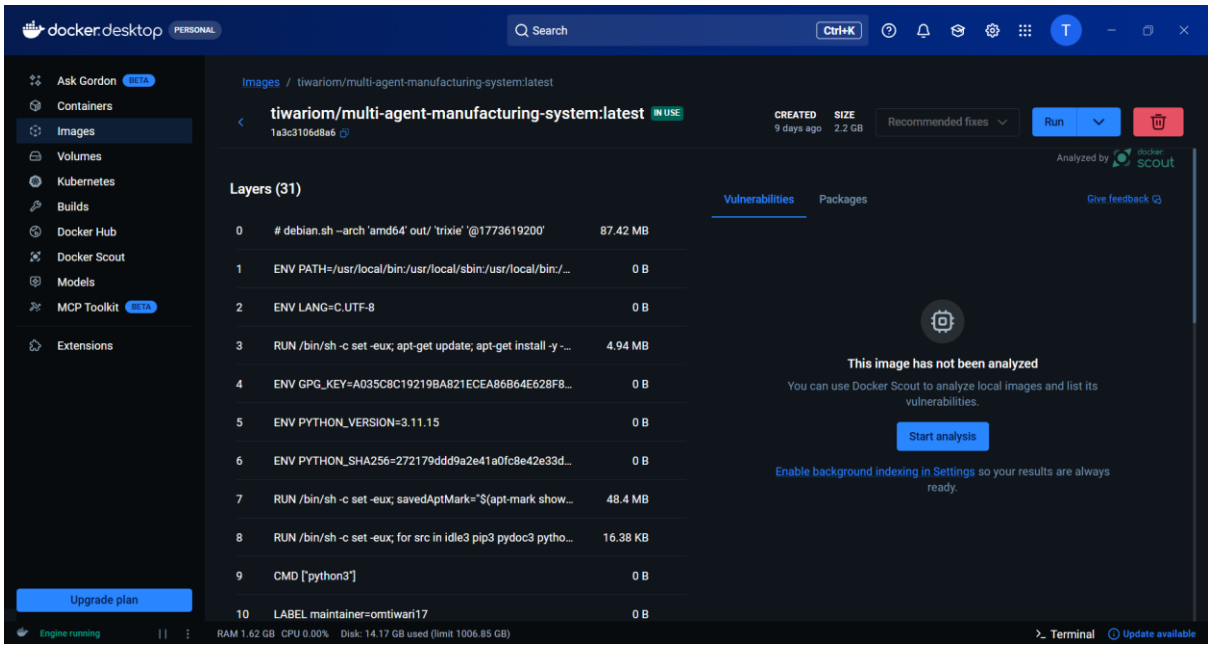
2.3 Tools & Technologies Used

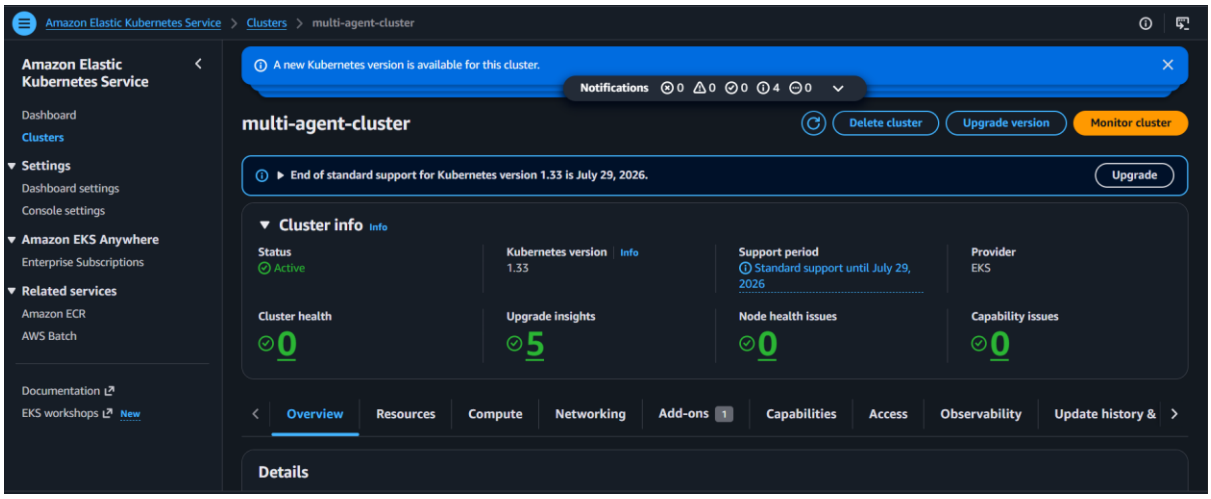
Category	Tool / Technology
Containerization	Docker, Docker Hub
Orchestration	Kubernetes, AWS EKS
CI/CD	GitHub Actions
Cloud	AWS EKS, AWS EC2, AWS Load Balancer, AWS VPC
Infrastructure	AWS IAM, AWS Auto Mode
Application	Python, Streamlit, CrewAI, Google Gemini LLM
Version Control	Git, GitHub

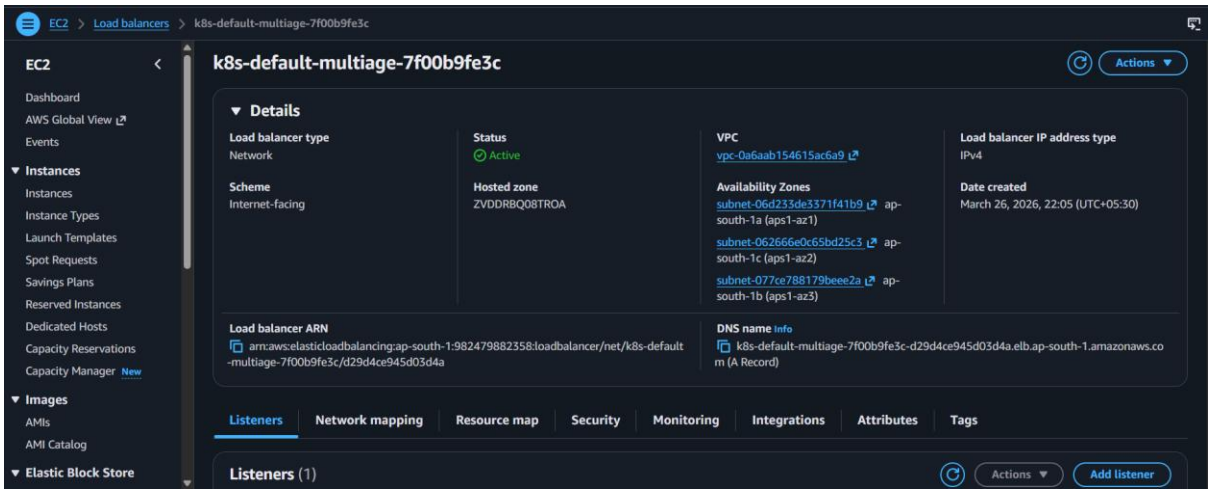
3. Results & Output

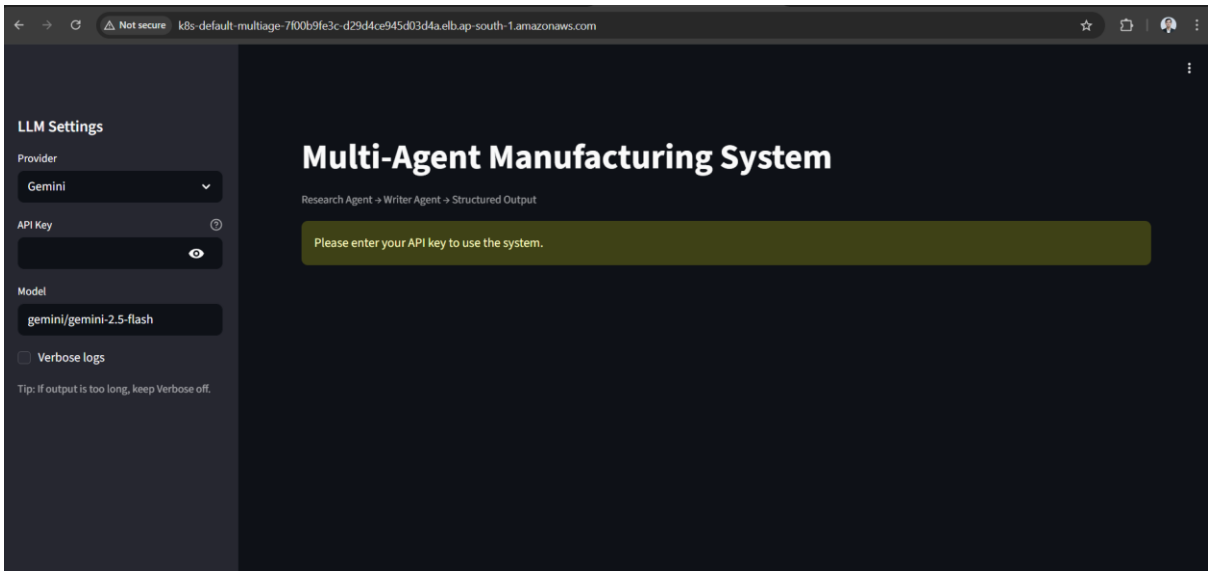
3.1 Screenshots / Outputs




docker desktop
Search
Ask Gordon
Containers
Images
Volumes
Kubernetes
Builds
Docker Hub
Docker Scout
Models
MCP Toolkit
Extensions
Upgrade plan
Images / tiwariom/multi-agent-manufacturing-system:latest
tiwariom/multi-agent-manufacturing-system:latest
IN USE
CREATED 9 days ago
SIZE 2.2 GB
Recommended fixes
Run
Layers (31)
0 # debian.sh --arch 'amd64' out/ 'trixie' '@1773619200' 87.42 MB
1 ENV PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/... 0 B
2 ENV LANG=C.UTF-8 0 B
3 RUN /bin/sh -c set -eux; apt-get update; apt-get install -y -... 4.94 MB
4 ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8... 0 B
5 ENV PYTHON_VERSION=3.11.15 0 B
6 ENV PYTHON_SHA256=272179ddd9a2e41a0fc8e42e33d... 0 B
7 RUN /bin/sh -c set -eux; savedAptMark="$(apt-mark show... 48.4 MB
8 RUN /bin/sh -c set -eux; for src in idle3 pip3 pydoc3 pytho... 16.38 KB
9 CMD ["python3"] 0 B
10 LABEL maintainer=omtiwari17 0 B
Vulnerabilities
Packages
Give feedback
This image has not been analyzed
You can use Docker Scout to analyze local images and list its vulnerabilities.
Start analysis
Enable background indexing in Settings so your results are always ready.
Engine running
RAM 1.62 GB CPU 0.00% Disk: 14.17 GB used (limit 1006.85 GB)
Terminal
Update available


Amazon Elastic Kubernetes Service > Clusters > multi-agent-cluster
Amazon Elastic Kubernetes Service
Dashboard
Clusters
Settings
Dashboard settings
Console settings
Amazon EKS Anywhere
Enterprise Subscriptions
Related services
Amazon ECR
AWS Batch
Documentation
EKS workshops New
A new Kubernetes version is available for this cluster.
Notifications
multi-agent-cluster
Delete cluster
Upgrade version
Monitor cluster
End of standard support for Kubernetes version 1.33 is July 29, 2026.
Upgrade
Cluster info Info
Status Active
Kubernetes version Info 1.33
Support period Standard support until July 29, 2026
Provider EKS
Cluster health 0
Upgrade insights 5
Node health issues 0
Capability issues 0
Overview
Resources
Compute
Networking
Add-ons 1
Capabilities
Access
Observability
Update history &
Details


EC2 > Load balancers > k8s-default-multiage-7f00b9fe3c
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
k8s-default-multiage-7f00b9fe3c
Actions
Details
Load balancer type Network
Status Active
VPC vpc-0a6aab154615ac6a9
Load balancer IP address type IPv4
Scheme Internet-facing
Hosted zone ZVDDRBQ08TROA
Availability Zones
subnet-06d233de3371f41b9 ap-south-1a (aps1-az1)
subnet-062666e0c65bd25c3 ap-south-1c (aps1-az2)
subnet-077ce788179beee2a ap-south-1b (aps1-az3)
Date created March 26, 2026, 22:05 (UTC+05:30)
Load balancer ARN
arn:aws:elasticloadbalancing:ap-south-1:982479882358:loadbalancer/net/k8s-default-multiage-7f00b9fe3c/d29d4ce945d03d4a
DNS name Info
k8s-default-multiage-7f00b9fe3c-d29d4ce945d03d4a.elb.ap-south-1.amazonaws.com (A Record)
Listeners
Network mapping
Resource map
Security
Monitoring
Integrations
Attributes
Tags
Listeners (1)
Actions
Add listener



3.2 Reports / Dashboards / Models

Deployment State- kubectl get all

NAME	READY	STATUS
pod/multi-agent-manufacturing-system-xx	1/1	Running
pod/multi-agent-manufacturing-system-xx	1/1	Running

NAME	TYPE	EXTERNAL-IP
service/multi-agent-manufacturing-service	LoadBalancer	k8s-default-xx.elb.ap-south-1.amazonaws.com

NAME	READY	UP-TO-DATE
deployment.apps/multi-agent-manufacturing-system	2/2	2

NAME	DESIRED	READY
replicaset.apps/multi-agent-manufacturing-system-xxx	2	2

GitHub Actions Pipeline- 9 Stages

Stage	Status
Checkout repository	Passed
Login to Docker Hub	Passed
Build Docker image	Passed
Push Docker image	Passed
Configure AWS credentials	Passed
Update kubeconfig for EKS	Passed
Deploy to Kubernetes	Passed
Verify deployment rollout	Passed
Show running pods	Passed

3.3 Key Outcomes

- Application successfully containerized and deployed on AWS EKS in Mumbai region
- Fully automated CI/CD pipeline operational zero manual deployment steps required
- Zero-downtime rolling updates verified app remained accessible throughout pod replacement
- Docker image publicly available on Docker Hub
- Application accessible globally via AWS Load Balancer public URL
- Total AWS infrastructure cost kept within $1–2 using free credits by deleting cluster when not in use

4. Conclusion

This project successfully demonstrates a complete production-grade DevOps implementation for deploying a Python Streamlit application onto AWS EKS. Starting from a locally running multi-agent AI system, we built a fully automated deployment pipeline that containerizes the application using Docker, orchestrates it with Kubernetes, and delivers it to the cloud through GitHub Actions CI/CD all triggered by a single git push.

The key learning from this project is the practical understanding of how containerization, Kubernetes orchestration, cloud infrastructure, and CI/CD pipelines work together as a

unified system. The two-repository architecture keeping application code and DevOps infrastructure completely separate proved to be a clean and maintainable approach. The health-check-driven rolling update strategy demonstrated how production systems achieve zero downtime in real-world deployments.

5. Future Scope & Enhancements

- **HTTPS support-** configure SSL certificate via AWS Certificate Manager with a custom domain for secure connections
- **Horizontal Pod Autoscaler-** automatically scale pod count based on CPU and memory usage during peak traffic
- **Persistent Storage-** integrate AWS S3 or EFS to persist agent run artifacts across pod restarts
- **Monitoring and Alerting-** integrate Prometheus and Grafana for real-time pod metrics, resource usage dashboards, and deployment alerts
- **Multi-environment pipeline-** add staging and production environments with separate Kubernetes namespaces and approval gates
- **Helm Charts-** package Kubernetes manifests as Helm charts for easier versioning and environment-specific configuration management
- **Private container registry-** migrate from Docker Hub to AWS ECR for tighter security and faster image pulls within AWS infrastructure

6. References

- **Kubernetes Documentation-** https://kubernetes.io/docs
- **AWS EKS Documentation-** https://docs.aws.amazon.com/eks
- **Docker Documentation-** https://docs.docker.com
- **GitHub Actions Documentation-** https://docs.github.com/en/actions
- **Streamlit Documentation-** https://docs.streamlit.io
- **CrewAI Documentation-** https://docs.crewai.com
- **Google Gemini API-** https://ai.google.dev/docs
- **Multi-agent-system Repo-** https://github.com/omtiwari17/Multi-agent-system
- **Docker Hub Image-** https://hub.docker.com/r/tiwariom/multi-agent-manufacturing-system

7. GitHub

https://github.com/omtiwari17/Deploy-Manufacturing-App-to-Kubernetes